# TP3: LiDAR

## ITI3: Capteurs

Robin Condat, Antoine Vaugeois

## 1 Objectifs de la séance

Aujourd'hui, un énième capteur: le LiDAR. Et plus particulièrement un LiDAR 2D 360° (on dirait un nom de figure de skate ou snowboard). Au programme, on va scanner toute la pièce, mesurer la précision du capteur et arrêter des bandits.

### 1.1 Matériel nécessaire

- Une Raspberry Pi 4
- Un RPLiDAR A1M8 (`https://www.slamtec.com/en/lidar/a1`)
- Un buzzer Grove v1.3 (`https://wiki.seeedstudio.com/Grove-Buzzer/`)

### 1.2 Packages Python utilisés

- *rplidar-roboticia*: Pour contrôler le LiDAR.
- *matplotlib*: Pour tracer des beaux graphiques :D
- *csv*: Pour enregistrer les données dans de beaux tableaux CSV

## 2 C'est quoi un LiDAR 2D ?

Avant de commencer à toute berzingue, on va déjà voir comment fonctionne un LiDAR 2D. Vous avez vu en cours le fonctionnement d'un LiDAR 3D. Sa version 2D est similaire, mais au lieu d'avoir plusieurs bandes (32, 64, 128, etc.), il n'en a qu'une seule. De ce fait, les mesures se feront uniquement dans un plan.

- Dans quel plan en 2D sont effectués les mesures ?
- Sachant que la tête du LiDAR tourne constamment, comment l'envoi et la réception du signal lumineux se fait-il ? (pour cela, n'hésitez pas à regarder la documentation et son joli schéma :D )
- Comment la distance est calculée à partir du signal lumineux ?

# 3 Mise en place

Cette semaine, la mise en place est super simple: il suffit de brancher en USB le LiDAR et c'est réglé. Enfin... Quasi...

- Téléchargez le code disponible sur Moodle, décompressez l'archive zip et renseignez le chemin de votre dossier dans la variable d'environnement `PYTHONPATH` :

```
export PYTHONPATH=/chemin/vers/votre/dossier/TP:$PYTHONPATH
```

- Après activation de l'environnement de travail `env_capteurs`, installez le package `rpi-lgpio` :

```
pip install rpi-lgpio
```

- Identifiez les ports USB utilisés sur votre Raspberry Pi en listant les noeuds de périphériques via la commande suivante:

```
ls /dev/*USB*
```

- Dans le fichier `config/constants.py` précisez le noeud de périphérique où communique le LiDAR en chaîne de caractères (`PORT_LIDAR`)

# 4 You spin me right round, baby, right round

Grâce au package python *rplidar-roboticia*, on va pouvoir obtenir facilement des premières mesures. Commençons déjà par un premier scan 360°. Le LiDAR va tourner sur 360° et effectuer plusieurs mesures (points) contenu dans un seul scan. Ce scan comprend un certain nombre de mesures, correspondant à différentes distances estimées à plusieurs angles (entre 0° et 360°).

- A l'aide de la documentation, définissez les portées maximale et minimale du LiDAR (en millimètres). Modifiez les constantes `LIDAR_MIN_RANGE` et `LIDAR_MAX_RANGE` dans le fichier `config/constants.py`
- Exécutez le script `scripts/single_scan.py` pour effectuer un premier scan rapide à 360°

⇒ Combien de mesures contient le scan ?

⇒ Quelles informations avez-vous pour chaque mesure ? Dans quel format ? Avec quelles unités de mesure ?

C'est bien joli les mesures comme ça, mais on ne voit pas grand chose... On va les transformer pour pouvoir tracer un nuage de points autour de notre LiDAR en 2D. C'est une représentation appelée Bird's Eye View (ou BEV) : une vue projeté depuis le ciel.

- Complétez la fonction `polar_to_cartesian` du fichier `sensors/LIDAR/lidar.py`, pour convertir les mesures dans un plan cartésien 2D. On considèrera que x=0 aux angles 0° et 180°, et y=0 aux angles 90° et 270°

  *Note: Attention, dans quel sens tourne le LiDAR ?*

⇒ Où se situe l'axe à 0° au niveau du LiDAR ?

- Décommentez l'appel de la fonction `plot_single_scan` dans le fichier `scripts/single_scan.py` et visualisez les mesures effectuées.

  *Note: si le résultat n'est pas satisfaisant, n'hésitez pas à surélever votre LiDAR, histoire d'avoir moins d'objets interférants*

Lors d'un scan LiDAR, on obtient un certain nombre de mesures, mais il se peut que beaucoup d'entre elles soient peu intéressantes en fonction de notre tâche. On peut alors filtrer les mesures obtenues selon plusieurs facteurs.

- Complétez la fonction `filter_quality` dans `sensors/LIDAR/lidar.py` pour filtrer les points de mauvaise qualité. Testez votre code en exécutant le script `scripts/single_scan.py` et en sélectionnant différents seuils de qualité via la variable `QUALITY_THRESHOLD` dans le fichier `config/parameters.py`)

- Complétez la fonction `filter_angle` dans `sensors/LIDAR/lidar.py` pour sélectionner uniquement les points se situant dans un intervalle angulaire prédéfini. Testez votre code en exécutant `scripts/single_scan.py` et en définissant différents intervalles angulaires (avec `MIN_DETECTION_ANGLE` et `MAX_DETECTION_ANGLE` dans le fichier `config/parameters.py`)

  *Note: pour détecter dans la section angulaire 90°-270°, on définira le minimum à 90° et le maximum à 270°. Si on veut détecter dans la section angulaire 270°-90°, on définira le minimum à 270° et le maximum à 90°*

- Complétez la fonction `filter_range` dans `sensors/LIDAR/lidar.py` pour sélectionner uniquement les points se situant dans une range de distance prédéfinie. Testez votre code en exécutant `scripts/single_scan.py` et en définissant différents portées minimales (avec `MIN_DETECTION_RANGE` et `MAX_DETECTION_RANGE` dans le fichier `config/parameters.py`)

  *Note: on différencie ici la portée physique du LiDAR (`LIDAR_MIN_RANGE` et `LIDAR_MAX_RANGE`), de la portée de détection (`MIN_DETECTION_RANGE` et `MAX_DETECTION_RANGE`)*

- Si tout est ok de votre côté, exécutez maintenant le script *stream_scan.py* et visualisez le résultat. N'hésitez pas à bouger votre LiDAR, ou à vous déplacez vous-même pour observer les variations de mesure.

# 5 Au millimètre près ?

L'objectif du LiDAR 2D est avant tout de mesurer précisément des distances. Avec nos précédentes visualisations, on a pu constater que les mesures étaient cohérentes, mais est-ce que les valeurs de distances sont correctes ?

- A partir de la documentation (et du site du constructeur), déterminez:
  - La fréquence d'acquisition du LiDAR (on utilise ici celle par défaut)
  - La précision des mesures de distances
  - La résolution des mesures de distances
  - La résolution angulaire

⇒ Quelle est la différence entre la précision et la résolution ?

- Complétez `measure_distance` dans le script `scripts/measure_distance.py`, afin d'estimer la distance moyenne et l'écart-type dans un intervalle d'angle très restreint ($\approx 5°$). Affichez ensuite (via le code) le nombre de points, la distance moyenne et l'écart-type pour chaque scan.

- Exécutez le script `scripts/measure_distance.py` et placez un objet à surface lisse à 25 cm du LiDAR, dans l'axe de mesure. L'estimation de la distance est-elle correcte ?

- Placez maintenant votre LiDAR face à un mur de la salle. Effectuez plusieurs mesures (enregistrées dans des fichiers CSV) en variant la distance face au mur (1m, 2m, 5m etc.), l'intervalle d'angle de détection, l'inclinaision du mur (en bougeant votre LiDAR tout en conservant la distance).

- Répétez ces opérations mais face à la fenêtre de la salle.

⇒ Comment évolue la précision des mesures du LiDAR selon les différents paramètres testés ? Quel(s) type(s) d'erreur(s) pouvons-nous constater ? Vous pouvez justifier vos affirmations à partir des mesures réalisées, ainsi que des schémas si vous le souhaitez.

# 6 WEEEU WEEEU, CECI EST UNE DESCENTE DE POLICE !

Votre mission, si vous l'acceptez, est de protéger la salle de TP des intrusions (majoritairement des enseignants qui n'ont pas réservé la salle et qui doivent finir urgemment la prépa de leurs TPs...). Pour cela, vous avez à votre disposition votre LiDAR 2D et un buzzer au bruit épouvantable, repoussant n'importe quel malotru. A vous de concevoir un détecteur de mouvement.

- Dans un premier temps, modifiez la fonction `proximity_monitor` dans le fichier `scripts/proximity_sensor.py` pour transformer le LiDAR en un détecteur de proximité. Le buzzer sonnera lorsque le LiDAR détectera, dans un intervalle d'angle précisé (ça peut être 360°), un objet proche (avec une distance en dessous d'un seuil que vous définirez). Exécutez ensuite le script et testez votre réalisation.

- Pour ceux qui aiment le challenge, modifiez la fonction `motion_monitor` dans le fichier `scripts/motion_sensor.py` pour créer un détecteur de mouvement. Pour cela, vous comparerez le scan actuel avec le précédent, et l'alarme sonnera seulement si un mouvement significatif est detecté. Afin de limiter l'impact des erreurs de détection, les mesures sont regroupées en sections angulaires (définie par un nombre de section *num_bins*), et on compare entre 2 scans la moyenne de distances obtenues dans chacunes des sections. N'hésitez pas à mettre un seuil de distance afin de faciliter la tâche... Exécutez ensuite le script et testez votre réalisation.